

PART 1

A)

1. superkeys: EmpID, SSN, Email, phone, (empid,name), (SSN, department)
2. candidate: EmpID, SSN, Email, phone
3. Primary key: EmpID, because it is compact, numeric, won't change after time
4. No, because it is impossible, but it can be changed by the time

B)

1.1.

Student_ID, CourseCode, Section, Semester, Year

1.2.

Student_ID: Identifies student

CourseCode: Identifies course

Section: Identifies course sections

Semester Year: allows student to retake or enroll in the same course diff academic terms

1.3.

if the system includes any other unique identifiers for student

2.

Student.AdvisorID -> Professor.ProfID

Department.ChairID -> Professor.ProfID

Course.DepartmentCode -> Department.DeptCode

Enrollment.StudentID -> Student.StudentID

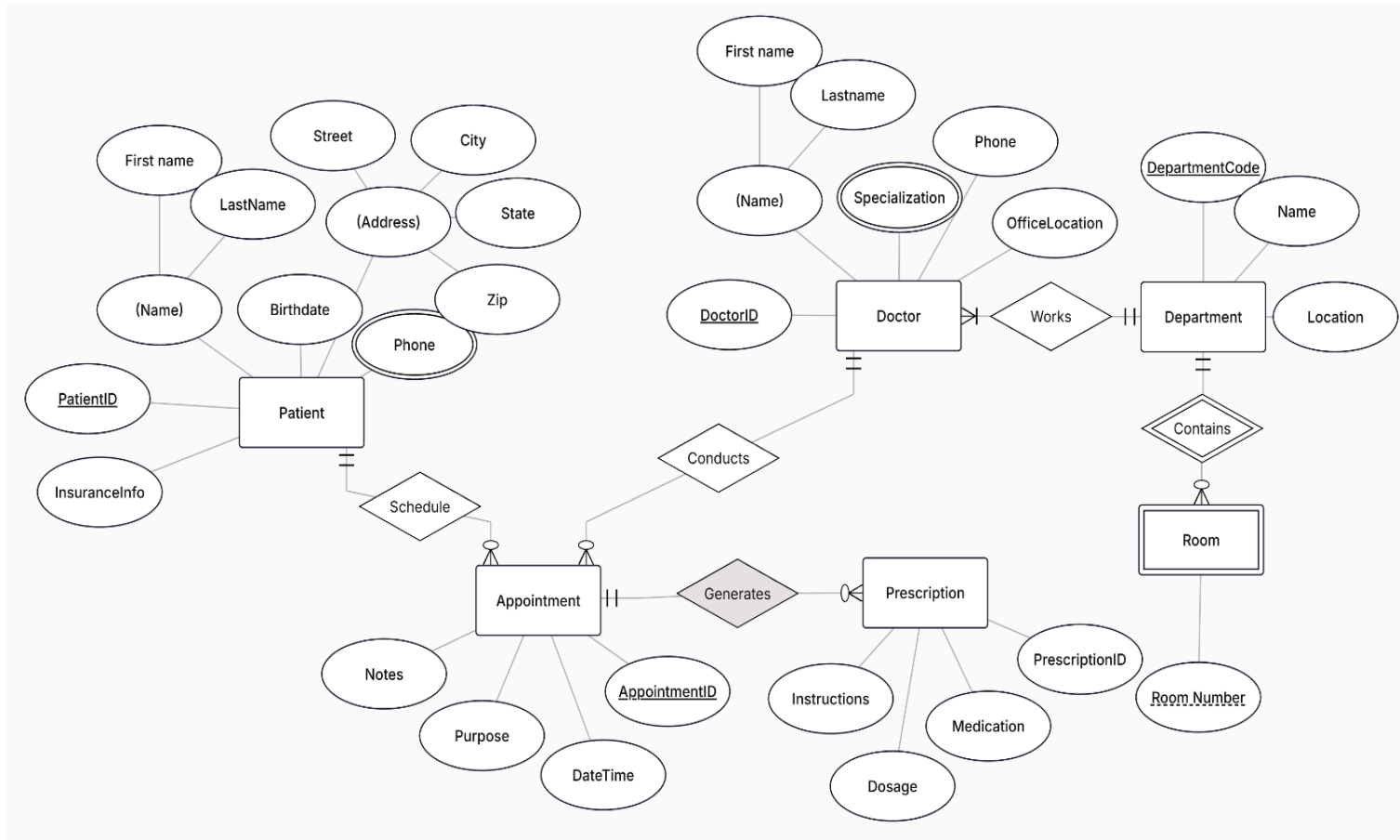
Enrollment.CourseID -> Course.CourseID

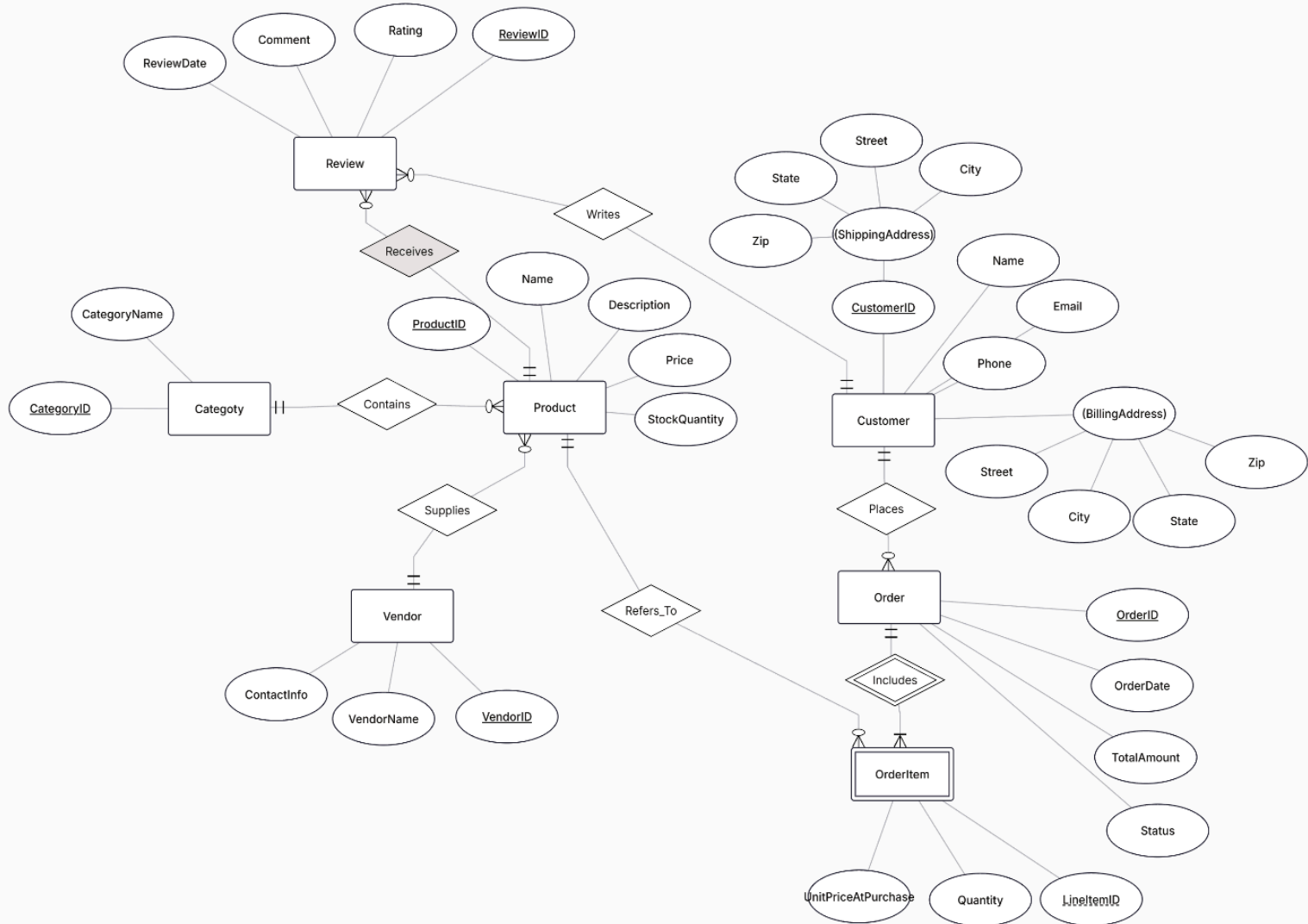
PART 2

Strong entities: Patient, Doctor, Department, Appointment, Prescription

Weak e.: Room

primary keys: PatientID, DoctorID, DepartmentCode, (DepartmentCode, RoomNumber), AppointmentID, PrescriptionID





PART 4

4.1

1)

StudentID -> StudentName, StudentMajor

ProjectID -> ProjectTitle, ProjectType, SupervisorID

SupervisorID -> SupervisorName, SupervisorDept

(StudentID, ProjectID) -> Role, HoursWorked, StartDate, EndDate

2)

redundancy: supervisor and student data repeated for every project assignment

update anomaly: Changing supervisors department update multiple rows in table, so it cause data instability

insert anomaly: a new supervisor cant be added

delete anomaly: removing a students last project record deletes supervisors details

3)

1nf: all values are atomic

4)

2nf:
primary key: StudentID, ProjectID
Student(StudentID, StudentName, StudentMajor)
Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)
StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5)
3nf:
Student(StudentID, StudentName, StudentMajor)
Supervisor(SupervisorID, SupervisorName, SupervisorDept)
Project(ProjectID, ProjectTitle, ProjectType, SupervisorID)
StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

4.2
1. primary key:
(StudentID, TimeSlot) or (StudentID, CourseID)

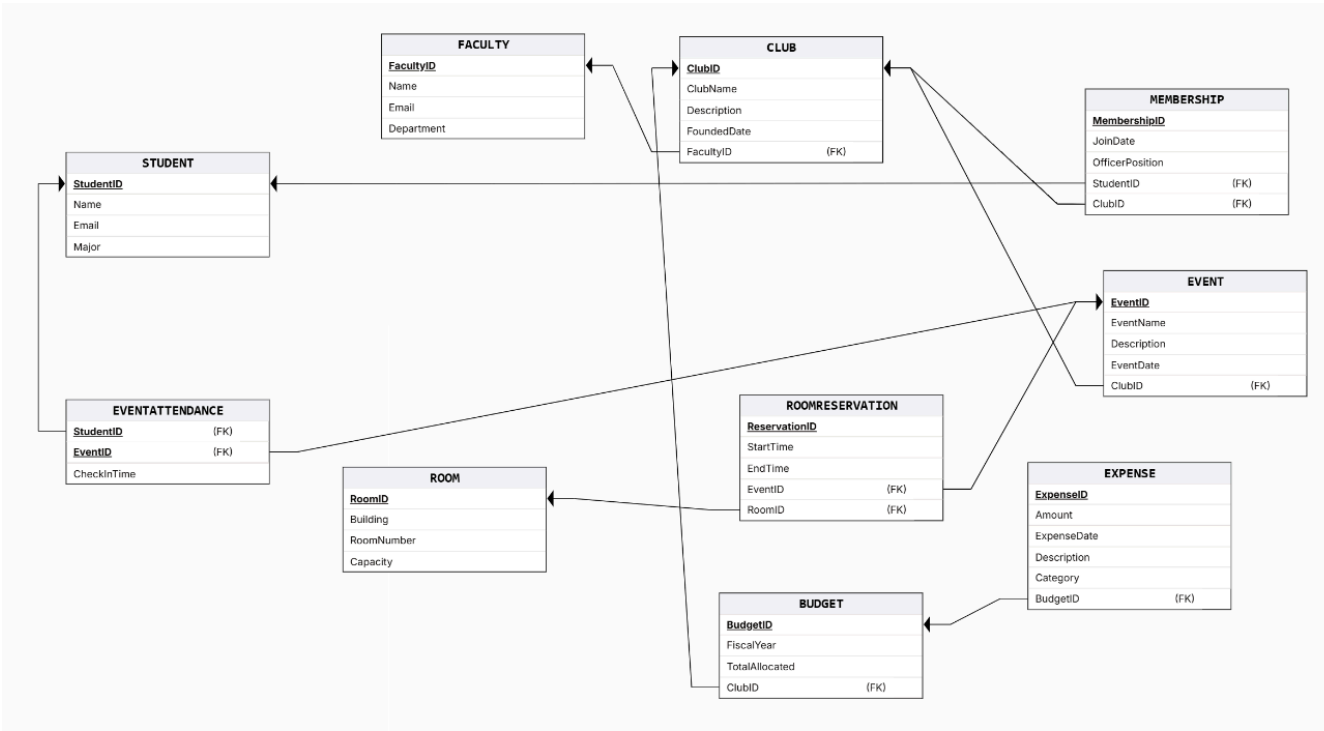
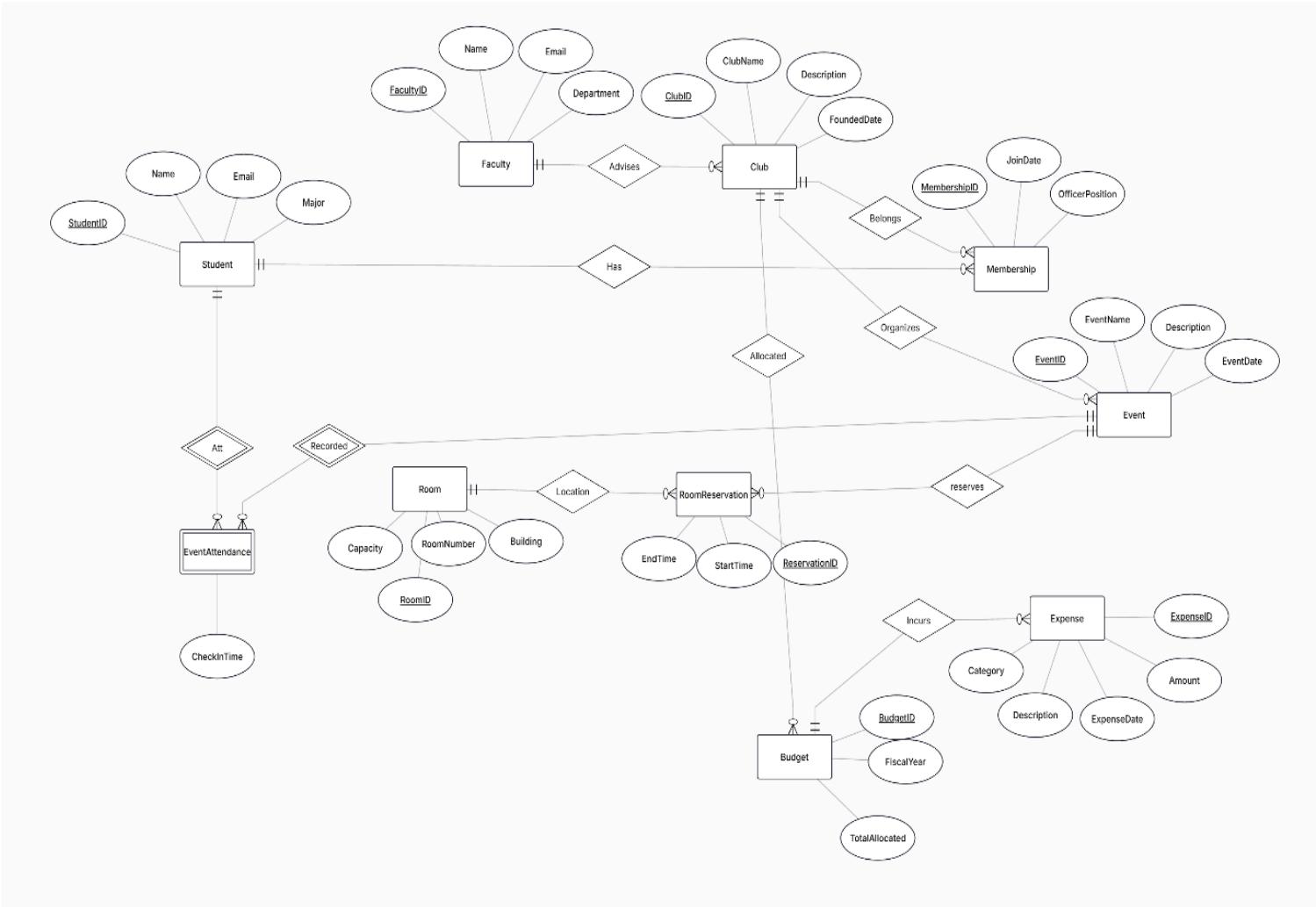
2.
StudentID -> StudentMajor
CourseID -> CourseName
InstructorID -> InstructorName
Room -> Building
CourseID, Timeslot -> InstructorID, Room

4.3
Not in BCNF. Some determinants are not candidate keys of the table

4.4
Student(StudentID, StudentMajor)
Course(CourseID, CourseName)
Instructor(InstructorID, InstructorName)
RoomLocation(Room, Building)
SectionSchedule(CourseID, TimeSlot, InstructorID, Room)
Enrollment(StudentID, CourseID, TimeSlot)

4.5
Decomposition dont lose any data. But because of this cross-table dependencies requires database triggers

PART 5



5.3

Create a separate Officer role entity with an associative table linking Student, Club and Role.

Store OfficerPosition as an optional attribute (nullable column) directly inside the Membership table.

Since every club officer is inherently a club member, placing the OfficerPosition field inside Membership avoids creating an additional junction table. This reduces the number of JOIN operations required when querying club rosters while keeping the database in Third Normal Form

5.4

List all upcoming club events scheduled for the next week, displaying the event name, organizing club, start/end times, and the reserved building and room number.

Calculate the remaining available budget for every club in the current fiscal year by subtracting the sum of all recorded expenses from the total allocated budget.

Retrieve a list of all students currently holding officer positions in a specific club, including their full names, emails, and assigned position titles